

Computer Architecture Final Project

Bilateral Filter on Scalar RISC-V, RVV, and CUDA

組員

314510148 翁瑄妤

314510169 黃麗穎

314580081 蔡程安

目錄

摘要.....	3
1. 題目與目標	3
2. 環境與硬體平台	3
2.1 Host Hardware and OS	3
2.2 RISC-V / gem5 Environment	4
2.3 CUDA Environment	4
3. 檔案與執行方式	5
4. 演算法內容	6
5. Correctness Verification	6
6. 各 Part 實現方法	7
6.1 Part 1 - Scalar Baseline	7
6.2 Part 2 - RVV Vector Reduction	7
6.3 Part 3 - SIMD-like RVV Parallelization	7
6.4 Part 4 - CUDA SIMT Implementation	8
6.5 Part 5 - Multi-pattern GPU Parallelism	8
7. 模擬與執行結果	8
7.1 gem5 Results: Part 1 to Part 3	8
7.2 CUDA Results: Part 4 and Part 5	10
8. 討論與比較	11

摘要

我們題目選擇 Bilateral Filter 作為測試演算法，目標是用同一個影像處理核心觀察 scalar、vector、SIMD-like RVV、CUDA SIMT 與 multi-pattern GPU parallelism 的效能差異。Bilateral Filter 對每個 output pixel 都要掃描 9 x 9 window，並根據 spatial distance 與 intensity difference 重新計算權重，因此有明顯的 nested loops、重複 arithmetic operations，以及高度獨立的 output pixels。這些特性剛好能對應本次 final project 要求的五個部分。

我們使用 cyberpunk2077_in.txt 作為主要測資，image size 為 160 x 107，總共 17,120 pixels，radius = 4，window 共有 81 個 neighbors，iterations = 3，單一 pattern 的 inner-loop evaluations 為 4,160,160。Part 2 的 RVV reduction 讓 instruction count 減少 9.43%，但 cycle count 只減少 0.45%。Part 3 改成 across-output SIMD-like mapping 後，cycle count 比 Part 1 減少 62.42%，speedup 約 2.66x。CUDA Part 4 的 GPU kernel time 為 0.106496 ms，相對 native CPU reference speedup 為 149.21x。Part 5 一次處理 32 個 independent patterns，GPU kernel time 為 2.053120 ms，per-pattern time 為 0.06416 ms，相對 Part 4 per-pattern time 改善 1.66x。

報告中的 gem5 simulated time 只用來比較 Part 1 至 Part 3；CUDA kernel runtime 則只用來比較 Part 4、Part 5 與其 native CPU reference。

1. 題目與目標

Bilateral Filter 是 edge-preserving smoothing filter。一般 blur 或 average filter 只看空間鄰近關係，容易把邊界一起抹掉；Bilateral Filter 會同時考慮 neighbor 與 center pixel 的距離和灰階值差異，所以在平滑雜訊時仍能保留影像邊緣。

從 computer architecture 的角度來看，這個演算法很適合本次 project。第一，每個 pixel 的輸出都要累加 81 個 neighbors，符合 nested-loop summation 的要求。第二，neighbor 的計算只使用加、減、乘、除，能直接對應 scalar/RVV/CUDA 的 arithmetic operations。第三，各 output pixels 之間幾乎沒有 dependency，很適合改寫成 vector lanes 或 CUDA threads。

Table 1. Workload setting used in the latest run.

項目	設定
Algorithm	Bilateral Filter
Input image	cyberpunk2077_in.txt
Image size	160 x 107
Total pixels	17,120
Radius / window	radius = 4, window = 9 x 9
Window elements per pixel	81
Iterations	3
Single-pattern inner-loop evaluations	4,160,160
Boundary policy	clamp-to-edge

2. 環境與硬體平台

2.1 Host Hardware and OS

Part 4、Part 5 的 CUDA kernel 與 native CPU reference 都是在實體工作站上實際執行。Part 1 至 Part 3 的 RISC-V CPU 則是 gem5 模擬出來的 TimingSimpleCPU。

Table 2. Physical host hardware and CUDA software environment.

項目	內容
Physical platform	Course-provided x86 workstation with NVIDIA GPU
Operating system	Rocky Linux 8.7 (Green Obsidian)
Host CPU	Intel(R) Xeon(R) Gold 6230 CPU @ 2.10GHz
CPU topology	4 sockets, 20 cores/socket, 2 threads/core, 160 logical CPUs
CPU frequency range	800 MHz to 3900 MHz
Host CPU cache	L1d 32 KB, L1i 32 KB, L2 1024 KB, L3 28160 KB
GPU	NVIDIA L4
GPU architecture	Ada Lovelace
GPU memory	23034 MiB
NVIDIA driver	535.104.05
CUDA version from nvidia-smi	12.2 shown by nvidia-smi
CUDA container	CUDA Version 12.4.1
nvcc version	Cuda compilation tools, release 12.4, V12.4.131
GPU max clocks	SM 2040 MHz, memory 6251 MHz

2.2 RISC-V / gem5 Environment

Part 1 至 Part 3 在 gem5 RISC-V 模擬環境中執行。gem5 statistics 用來觀察 simulated time、instruction count、cycle count、CPI/IPC 與 cache miss rate。

Table 3. gem5 configuration summary.

項目	內容
Docker image	weisheng505/gem5-rvv-image:v1
Simulator	gem5 23.1.0.0
Compiler	riscv64-linux-gnu-g++
CPU model	TimingSimpleCPU
ISA	RISC-V RV64 + RVV
RVV setting	VLEN = 256 bits, ELEN = 64 bits
Memory size	256 MB
L1 I-cache	32 KB, 8-way, cache line = 32 B
L1 D-cache	32 KB, 8-way, cache line = 32 B
Observed stats	simSeconds, simInsts, numCycles, CPI, IPC, overallMissRate

L1 I-cache 與 L1 D-cache 的 32 KB 設定來自 gem5 command line 的 --l1i_size=32kB、--l1d_size=32kB，以及 m5out/config.ini 中的 size=32768。

```
# enter the provided gem5 Docker environment first
cd /workspace/p1 # or p2 / p3
make clean
make
```

2.3 CUDA Environment

Part 4 與 Part 5 在 CUDA container 中執行，使用 nvcc 編譯 GPU kernel。GPU runtime 使用 cudaEvent 量測 kernel-only time，不包含 input file loading、host-to-device copy、device-to-host copy 與 output file writing。CPU reference 使用 native C++ chrono 量測，目的是驗證 CUDA 版本相對 host CPU baseline 的加速效果。

Table 4. CUDA environment and timing method.

項目	內容
CUDA container runtime	CUDA Version 12.4.1
nvcc	Cuda compilation tools, release 12.4, V12.4.131
Compiler flags	nvcc -O2 -Xptxas -v
Target architecture	-arch=sm_89
GPU platform	NVIDIA L4
Timing method	cudaEvent kernel-only timing
CPU reference timing	native CPU chrono timing on Intel Xeon Gold 6230 workstation
P4 mapping	one CUDA thread = one output pixel
P5 mapping	2D grid: x = pixel index, y = pattern index

```
# enter the CUDA Docker environment first
cd /workspace/p4 # or p5
make clean
make
./main
./main_cpu
```

我們只在同一類量測之內做 speedup。Part 1 至 Part 3 比較 gem5 simulated statistics；Part 4、Part 5 比較 actual CUDA GPU kernel runtime 與 host CPU native runtime。

3. 檔案與執行方式

專案依照五個 parts 分開實作，讓每個 implementation 的 mapping 與效能行為可以獨立觀察。

Table 5. Project file organization.

路徑	內容	主要輸出
p1/	Scalar RISC-V baseline	output/scalar_out.txt
p2/	RVV vector reduction	output/rvv_out.txt
p3/	SIMD-like RVV across-output parallelization	output/simd_rvv_out.txt
p4/	CUDA SIMT single-pattern implementation + CPU reference	output/cuda_out.txt, output/cpu_native_out.txt
p5/	CUDA multi-pattern 2D-grid implementation + CPU reference	output/multi_cuda_out.txt, output/cpu_multi_out.txt
test/	Main input file	cyberpunk2077_in.txt
tools/	txt/png conversion utilities	preprocess_image_to_txt.py, txt_to_png.py

為了讓 RISC-V/gem5、RVV inline assembly 與 CUDA kernel 都使用同一份簡單且可重現的輸入，我們沒有在核心程式中直接讀取 PNG。PNG 本身包含壓縮、header、metadata 與影像格式解析，若在每個 part 都實作 PNG decoder，工作量會偏離 computer architecture 的重點，也會讓 gem5 靜態編譯與 CUDA reference 程式變得複雜。因此我們先把圖片轉成數值化的 txt input，再讓五個 parts 都讀取相同的 numeric image。

前處理使用 tools/preprocess_image_to_txt.py，將原始 RGB PNG 轉成 grayscale pixel values，輸出格式為第一行 width height，後面接 0 到 255 的 pixel values。本次輸入 cyberpunk2077_in.txt 的 header 為 160 x 107，因此程式可直接配置 17,120 個 float pixels，不需要處理 PNG 壓縮或色彩格式。

各 part 的 filter kernel 會輸出 txt 檔，內容是濾波後的 pixel values。後處理使用 tools/txt_to_png.py 將 output txt clamp/round 回 0 到 255 的 grayscale image，主要用於產生 PNG 圖片做 visual sanity check。前處理、後處理、file I/O 與 host-device memory copy 都不列入 kernel timing；報告中的 performance 比較只針對核心濾波運算。

Table 6. Image preprocessing and postprocessing flow.

Stage	Input	Output	Purpose
Preprocessing	raw PNG image	grayscale numeric txt	Remove PNG decoding from the measured programs and provide a simple numeric input.
Main computation	width, height, pixel values	filtered txt output	Run the same bilateral filter workload in scalar RISC-V, RVV, and CUDA.
Postprocessing	filtered txt output	grayscale PNG image	Convert numeric output back to an image for visual checking.
Verification	numeric output values	checksum / output statistics	Compare implementations without relying only on visual inspection.

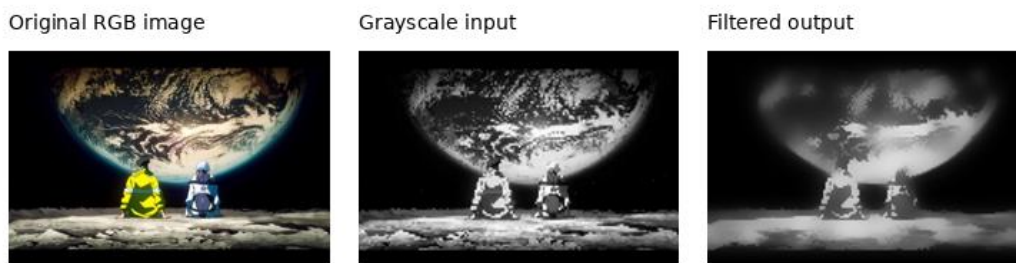


Figure 1. Original image, grayscale input, and filtered output used for visual sanity check.

4. 演算法內容

對每個 output pixel p ，Bilateral Filter 會掃描以 p 為中心的 9×9 window。每個 neighbor q 會產生一個 weight，weight 由 spatial weight 和 range weight 相乘得到。最後的 output 是 neighbor pixel values 的 weighted average。

```
spatial_weight(dx, dy) = 1 / (1 + SPATIAL_ALPHA * (dx*dx + dy*dy))
range_weight(center, neighbor) = 1 / (1 + RANGE_ALPHA * (neighbor - center)^2)
weight = spatial_weight * range_weight
output[p] = sum(weight * neighbor) / sum(weight)
```

我們在實作時用 rational approximation 取代 $\exp()$ ，range kernel 為 $1 / (1 + \text{RANGE_ALPHA} * \text{diff} * \text{diff})$ 。這個寫法保留 Bilateral Filter 中「像素值差越大，權重越小」的行為，也讓 inner loop 只需要基本 arithmetic operations，符合 scalar/RVV/CUDA 三種實作的共同基準。

Table 7. Bilateral filter parameters.

Parameter	Value
RADIUS	4
WINDOW_SIZE	9×9
KERNEL_ELEMS	81
SPATIAL_ALPHA	0.0600
RANGE_ALPHA	0.0009
Boundary policy	clamp-to-edge
Iterations	3

5. Correctness Verification

Correctness 使用三層檢查。第一，constant image test 將所有 pixels 設成 128，理論上 filter 後仍應該是 128，本次各版本輸出的 constant_image_max_abs_err 都是 0.00000000。第二，對實際影像輸出計算 rounded 8-bit checksum。第三，CUDA 版本使用 native CPU reference，可比對 GPU 與 CPU 的 checksum 與 output_sum。

Table 8. Correctness verification result.

Part	Output summary	Checksum / status
P1 Scalar	output_sum = 992332.832534	0x06a2ca9cc3a8d2f6
P2 RVV reduction	output_sum = 992332.815146	0x06a2ca9cc3a8d2f6
P3 SIMD-like RVV	output_sum = 992332.834205	0x06a2ca9cc3a8d2f6
P4 CUDA GPU	gpu_output_sum = 992332.837364	0x06a2ca9cc3a8d2f6
P4 native CPU	cpu_output_sum = 992332.838018	0x06a2ca9cc3a8d2f6
P5 GPU pattern 0	gpu_output_sum_pattern0 = 992332.837364	0x06a2ca9cc3a8d2f6
P5 CPU pattern 0	cpu_output_sum_pattern0 = 992332.838018	0x06a2ca9cc3a8d2f6
P5 all patterns	GPU/CPU all-pattern checksum match	0x550953d83cf909ad

各版本的 output_sum 有極小差異，主要來自 scalar order、vector reduction order 與 GPU floating-point execution order 不同。checksum 使用 rounded and clamped 8-bit pixel values，因此輸出的可視化結果與最終 pixel-level verification 仍一致。

6. 各 Part 實現方法

6.1 Part 1 - Scalar Baseline

Part 1 使用純 C/C++ scalar nested loops。外層依序走訪 y、x，每個 output pixel 再掃描 81 個 window elements。每個 neighbor 都獨立計算 diff、range_weight、weight、weighted_sum 與 weight_sum。

```
for each pixel (x, y):
    weighted_sum = 0
    weight_sum = 0
    for each of 81 neighbors:
        compute range_weight and weight
        weighted_sum += weight * neighbor
        weight_sum += weight
    dst[pixel] = weighted_sum / weight_sum
```

6.2 Part 2 - RVV Vector Reduction

Part 2 的 mapping 是「一個 output pixel 一次處理」，但把 9 x 9 window 的 neighbor summation 改成 RVV vector arithmetic。gem5 設定中的 VLEN = 256 bits，因此 FP32 最多一次處理 8 個 elements。81 個 neighbors 會切成 11 個 vector chunks，每個 chunk 使用 vector arithmetic 計算 weight 與 weighted value，再用 vfredusum.vs 分別得到 numerator chunk sum 與 denominator chunk sum。

由於 window 中經過 clamp 的 neighbor address 不一定連續，程式先把 81 個 neighbor values gather 到 temporary contiguous array，再用 vle32.v 做 unit-stride vector load。這樣可以滿足 Part 2 的 vector reduction 目標，但 gather 本身仍是 scalar overhead。

Table 9. Part 2 RVV reduction mapping.

Part 2 item	Implementation detail
Vector length	MAX_VL_FLOAT = 8
Chunks per pixel	ceil(81 / 8) = 11
Load type	unit-stride load, vle32.v
Arithmetic	vfsb, vfmul, vfadd, vfdiv
Reduction	vfredusum.vs for weighted_sum and weight_sum
Trade-off	instruction count decreases, but gather/reduction/setup overhead remains

6.3 Part 3 - SIMD-like RVV Parallelization

Part 3 改成 across-output mapping。每個 vector lane 對應一個 independent output pixel，等於一次處理 k 個 pixels。這裡 k 選擇 8，對應 VLEN = 256 bits 下的 8 個 FP32 lanes。每個 lane 自己累加自己的 weighted_sum 與 weight_sum，所以不需要 horizontal reduction。

此 mapping 需要 strided vector access，因為同一個 neighbor offset j 下，k 個 pixels 的 memory addresses 彼此相隔固定 stride。內部 pixels 可直接用 vector lanes 平行處理；邊界 pixels 需要 clamp，控制較複雜，因此保留 scalar path。本次每個 iteration 有 15,048 個 vectorized pixels 與 2,072 個 scalar boundary pixels。

Table 10. Part 3 SIMD-like RVV mapping.

Part 3 item	Implementation detail
k-way unrolling	k = 8 output pixels per vector group
Vector lane meaning	one lane = one independent output pixel

Reduction operation	not used
Memory access	strided vector load/store, vlse32.v / vsse32.v concept
Vectorized pixels per iteration	15,048
Scalar boundary pixels per iteration	2,072
Main benefit	avoid reduction overhead and use lanes for independent outputs

6.4 Part 4 - CUDA SIMT Implementation

Part 4 使用 CUDA SIMT，一個 CUDA thread 計算一個 output pixel。grid 的 x dimension 對應 flattened pixel index，block size 設為 256 threads，因此 17,120 個 pixels 需要 67 個 blocks。每個 thread 在自己的 9 x 9 window 內做 81 次 neighbor operations，iterations = 3。

nvcc 使用 -arch=sm_89。PTXAS 顯示 kernel 使用 20 registers/thread，stack frame 為 0 bytes，spill stores/loads 都是 0。這代表 register allocation 沒有產生 local memory spills，對 GPU kernel runtime 有利。

Table 11. Part 4 CUDA mapping and PTXAS result.

Part 4 item	Value
threads_per_block	256
cuda_blocks	67
active_threads_per_iter	17,120
launched_threads_per_iter	17,152
PTXAS registers/thread	20
PTXAS spills	0 stores / 0 loads
Timing	cudaEvent kernel-only

6.5 Part 5 - Multi-pattern GPU Parallelism

Part 5 把 GPU workload 從單一 pattern 擴充成 32 個 independent patterns。kernel 使用 2D grid，blockIdx.x 對應 pixel block，blockIdx.y 對應 pattern index。pattern 0 是原始輸入影像，pattern 1 至 pattern 31 是 deterministic variants，沒有使用 randomness。

這個設計讓 active threads 從 Part 4 的 17,120 增加到 547,840。對於現代 GPU 來說，單張 160 x 107 image 的 blocks 太少，容易無法填滿所有 SM；32 patterns 則提供更多 blocks 與 warps，讓 scheduler 在某些 warps 等待 memory 或 floating-point division 時仍有其他 warps 可執行。

Table 12. Part 5 CUDA multi-pattern mapping.

Part 5 item	Value
patterns	32
grid_x	67
grid_y	32
total blocks	2,144
active_threads_per_iter	547,840
launched_threads_per_iter	548,864
PTXAS registers/thread	25
PTXAS spills	0 stores / 0 loads

7. 模擬與執行結果

7.1 gem5 Results: Part 1 to Part 3

Part 1、Part 2、Part 3 都使用同一個 cyberpunk2077_in.txt，image size 皆為 160 x 107，所以 raw simSeconds、simInsts 與 numCycles 可以直接比較。

Table 13. gem5 raw statistics for the same input image.

Part	Mapping	simSeconds	simInsts	numCycles	D-cache miss rate
P1	Scalar	0.211487	148,846,525	422,974,244	0.001152
P2	RVV reduction	0.210532	134,811,600	421,064,168	0.000961
P3	SIMD-like RVV	0.079487	50,994,483	158,973,846	0.001901

Table 14. Derived gem5 metrics normalized by inner-loop evaluations.

Part	CPI	IPC	Inst / eval	Cycles / eval	Cycle speedup vs P1
P1	2.841680	0.351904	35.78	101.67	1.00x
P2	3.123353	0.320169	32.41	101.21	1.00x
P3	3.117471	0.320773	12.26	38.21	2.66x

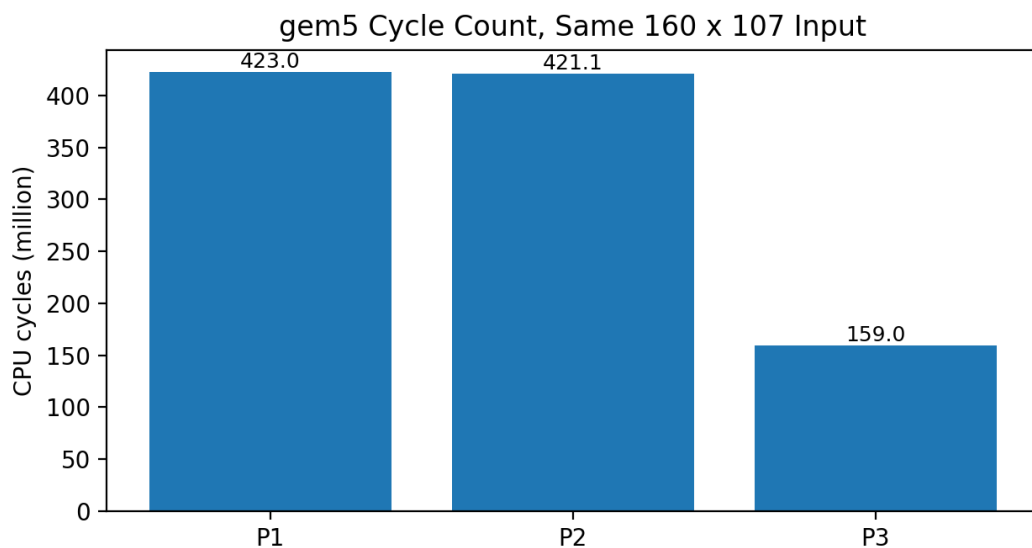


Figure 2. Cycle count from gem5. P3 has the clearest cycle reduction.

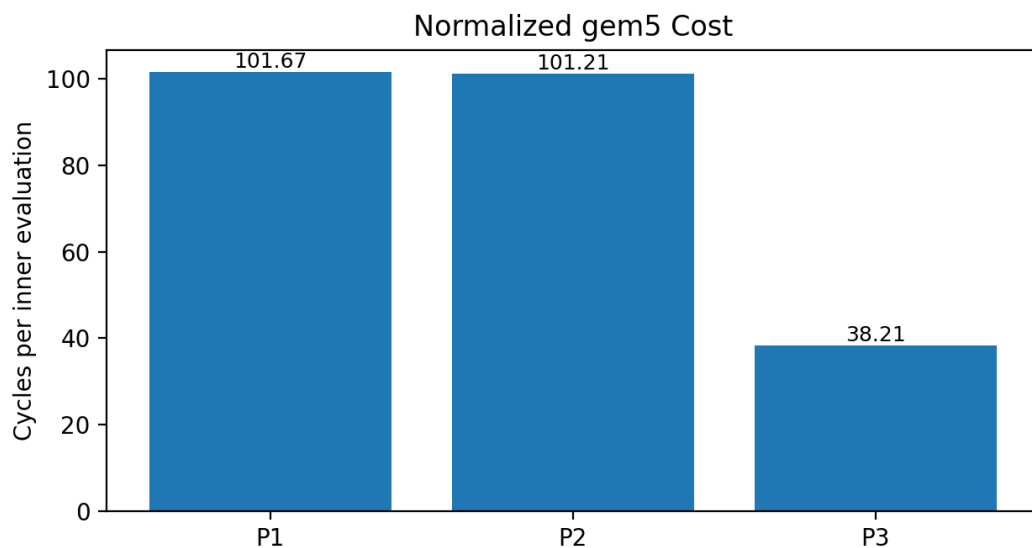


Figure 3. Normalized cycle cost per inner-loop evaluation.

Part 2 的 instruction count 比 Part 1 少 9.43%，但 cycles 只少 0.45%。這符合我們對 Part 2 的觀察：vector arithmetic 有減少部分指令，但每個 output pixel 仍需 gather 81 個 neighbors、執行 11 次 vector chunks、做兩個 vector reductions，pixel-level parallelism 沒有被打開。

Part 3 的 instruction count 比 Part 1 少 65.74%，cycle count 少 62.42%，相對 Part 1 speedup 約 2.66x，相對 Part 2 speedup 約 2.65x。D-cache miss rate 從 P2 的 0.000961 上升到 P3 的 0.001901，代表 strided access 確實增加 cache 壓力；但 P3 少掉 reduction overhead，且每個 vector instruction 對多個 output pixels 做有效工作，所以整體仍明顯變快。

從 memory behavior 看，三個 gem5 版本的 I-cache miss rate 都很低，表示 instruction footprint 不是主要瓶頸。D-cache miss rate 也低於 0.2%，所以主要差異不是由 cache miss 主導，而是由 inner loop mapping 決定。P1 的問題是所有 neighbor evaluation 都用 scalar loop 執行；P2 雖然把 neighbor arithmetic vectorize，但還是單一 output pixel 為中心；P3 把獨立 output pixels 展開成 vector lanes，才真正把 pixel-level parallelism 暴露給 RVV。

7.2 CUDA Results: Part 4 and Part 5

CUDA 結果使用 actual runtime，因此只與 native CPU reference 做比較。GPU time 是 cudaEvent 量到的 kernel-only time，CPU time 是 main_cpu.cpp 用 chrono 量到的 compute time。

Table 15. CUDA runtime and speedup over native CPU reference.

Part	Patterns	GPU kernel time	CPU reference time	Speedup	Throughput
P4	1	0.106496 ms	15.889981 ms	149.21x	39.06 G eval/s
P5	32	2.053120 ms	552.215774 ms	268.96x	64.84 G eval/s

Table 16. CUDA parallelism scale and checksum.

Part	Active threads / iter	Launched threads / iter	Grid	Per-pattern kernel time
P4	17,120	17,152	67 x 1	0.106496 ms
P5	547,840	548,864	67 x 32	0.064160 ms

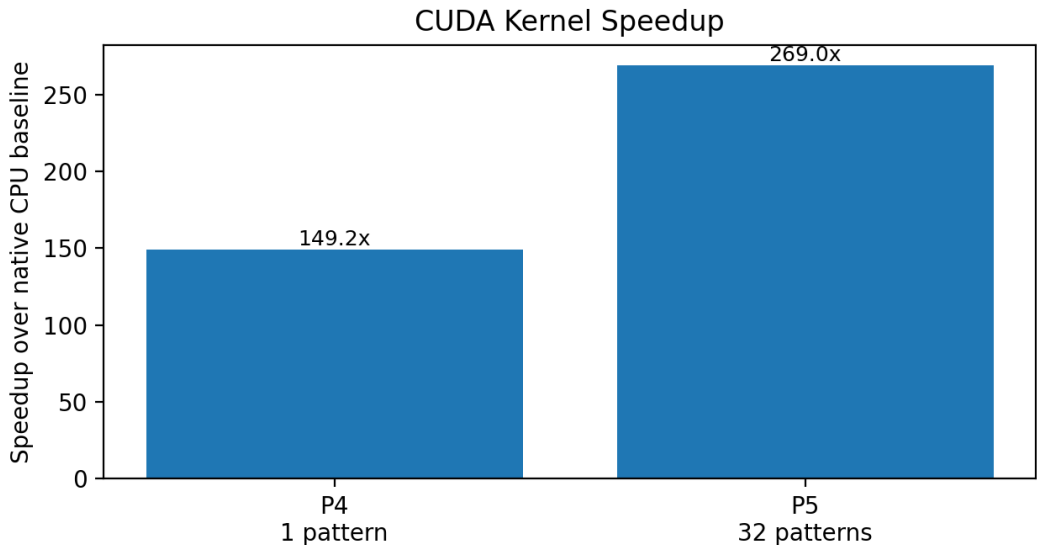


Figure 4. CUDA speedup over native CPU reference.

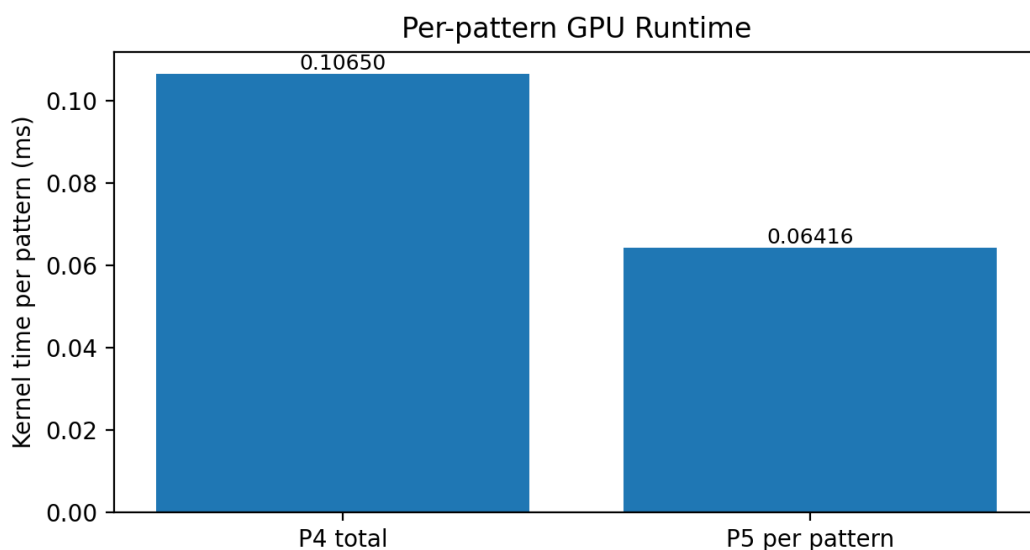


Figure 5. P5 improves per-pattern kernel time by increasing GPU workload size.

Part 4 的 kernel time 是 0.106496 ms，CPU native time 是 15.889981 ms，speedup 為 149.21x。Part 5 一次處理 32 個 patterns，總 GPU kernel time 變成 2.053120 ms，因為 workload 是 32 倍，總時間本來就會比 Part 4 長；公平比較應看 per-pattern time。P5 per-pattern time = 0.06416 ms，比 P4 的 0.106496 ms 快 1.66x。

這個結果的重點不是單純列出 ms，而是說明 GPU 是否有足夠 parallel work。P4 只有 67 個 blocks，對 NVIDIA L4 這類 GPU 來說 workload 偏小，kernel launch 後可排程的 warps 不多，latency hiding 空間有限。P5 把 pattern dimension 放進 grid.y，blocks 從 67 增加到 2,144，active threads 從 17,120 增加到 547,840。每個 pattern 的演算法完全獨立，因此這種擴張不需要同步或跨 pattern communication，能更有效填滿 GPU。

8. 討論與比較

8.1 為什麼 cycle reduction 與 instruction reduction 不完全相同？

Instruction count 只描述 commit 的指令數，cycle count 還會受到每種 instruction latency、memory stall、pipeline behavior、vector setup、reduction latency 與 cache miss 影響。Part 2 是最清楚的例子：RVV reduction 少掉約 9.43% instructions，但 cycles 只少約 0.45%。原因不是 RVV 沒有作用，而是這個 mapping 還保留了 per-pixel serial structure，且 81-element window 需要 11 個 chunks，每個 chunk 都有 vsetvli、vector division、兩次 reduction 與 scalar result extraction。

Part 3 則呈現另一種現象。它的 CPI 比 P1 高，但每個 instruction 做的有效工作較多，因此總 cycles 仍大幅下降。這代表看 vector 程式時不能只看 CPI；應該同時看 cycles per useful operation 或 cycles per inner evaluation。

8.2 當 k 愈大，performance 一定愈好嗎？

不一定。k 增加時，理想上可以讓一個 vector instruction 同時處理更多 output pixels，減少 loop overhead，提升 data parallelism。本次 RVV 設定下 FP32 max vl 是 8，所以 k=8 是自然的選擇。若 k 超過可用 vector lanes，就要拆成多個 vector groups，效益不會繼續線性增加。

更大的 k 也可能帶來 register pressure、更多 temporary vectors、較複雜的 boundary handling，以及 strided memory access 對 cache locality 的影響。當 memory behavior 或 register usage 開始限制執行時，增加 k 反而可能讓 performance 停滯或下降。

8.3 為什麼 Part 3 不需要 reduction operations ?

Part 2 的 vector lanes 放的是同一個 output pixel 的不同 neighbors，所以最後需要 horizontal reduction，把多個 lane 的 partial values 加成一個 scalar numerator 與 denominator。Part 3 的 vector lanes 放的是不同 output pixels，每個 lane 都代表一個獨立結果。若在 Part 3 使用 reduction，等於把不同 pixels 的結果加在一起，語意上是錯的。

Part 3 的取捨是少掉 reduction overhead，換來 strided access 與邊界處理成本。從結果看，這個取捨對本專案的 Bilateral Filter 是值得的。

8.4 Shared memory 在什麼情況下有效？

Shared memory 對 tile-based computation 最有效，尤其是同一個 block 內的 threads 會重複使用同一批 global memory data 時。Bilateral Filter 的相鄰 output pixels 有大量重疊 window，因此 P4 與 P5 目前的 shared mode 會把每個 pixel block 需要的 row tile 加上上下 halo rows 載入 dynamic shared memory，再讓 block 內 threads 重複讀取這些資料。

因此本次 CUDA kernel 已經有 shared-memory 路徑。P4 預設使用 row-tile shared-memory kernel，P5 也在每個 pattern 的 block 內使用相同策略；P5 的 grid.y 代表 pattern index，各 pattern 彼此獨立，不需要跨 pattern synchronization 或 shared-memory sharing。如果計算出的 shared_memory_bytes_per_blk 超過 device sharedMemPerBlock，程式會印出 warning 並 fallback 到 global mode。由於目前輸入只有 160 x 107，shared memory 的效益會受到 tile loading、boundary handling 與 __syncthreads() overhead 影響；但在更大的 image 或更大的 radius 下，重疊 window 的資料重用會更明顯，shared memory 比 global-only kernel 更可能帶來效能提升。

8.5 Block size 愈大，performance 一定愈好嗎？

不一定。block size 會影響每個 block 的 warps 數量，也會影響一個 SM 上能同時 resident 的 blocks 數量。threads_per_block = 256 時，每個 block 有 8 warps，搭配 20 registers/thread 或 25 registers/thread 沒有產生 spills，是一個合理設定。

若 block size 改成 512，單一 block 的 warps 變多，但每個 block 需要更多 registers，可能讓 resident blocks per SM 下降，降低 scheduling flexibility。若 block size 太小，例如 64，blocks 數量會上升，但每個 block 的 warps 太少，block scheduling overhead 與 occupancy 表現未必較好。因此 block size 需要用 occupancy、active warps per SM、register usage 與 kernel time 一起判斷。

8.6 Occupancy 與 latency hiding

GPU 的 latency hiding 依賴大量 ready warps。當某個 warp 遇到 memory latency 或 floating-point division latency，SM scheduler 可以切到其他 ready warps，讓 execution units 不要空等。Occupancy 越高通常代表可排程的 warps 越多，但 occupancy 不是唯一指標；如果 kernel 已經受限於 arithmetic throughput 或 memory bandwidth，單純提高 occupancy 不一定能繼續提升 runtime。

Part 4 只有 67 blocks，對現代 GPU 來說 workload 偏小。Part 5 變成 $67 \times 32 = 2,144$ blocks，每個 block 8 warps，單次 kernel launch 共有 17,152 launched warps。這就是 Part 5 per-pattern time 比 Part 4 更低的主要原因。

8.7 Is the CUDA kernel memory-intensive or compute-intensive?

實作的 kernel 比較偏 compute-intensive。每個 neighbor 至少需要 diff、diff squared、range denominator、division、weight multiplication、weighted sum 與 weight sum。影像大小只有 17,120 pixels，單張 FP32 image 約 67 KB，資料量相對小；而每個 pixel 要做 81 neighbors x 3 iterations，arithmetic density 高。

Global memory access 當然仍存在，特別是 Part 5 同時處理 32 patterns 時 input footprint 變大；但從 algorithm structure 看，floating-point division 與大量 multiply/add 是主要成本。這也解釋了為什麼增加 patterns 後可以更好 hide latency，per-pattern time 下降。

8.8 為什麼 Part 5 比 Part 4 更適合 GPU？

Part 4 的 parallelism 只來自單張 image 的 17,120 pixels。對 CPU 來說這已經很多，對 GPU 來說卻偏小，因為 kernel launch 後能提供的 blocks 和 warps 不夠多。Part 5 把 independent patterns 也放進 grid.y，讓 GPU 同時看到 32 倍的 independent work。這種設計更符合 GPU 喜歡大量 threads/warps 的特性。

增加 pattern 數量不會永遠線性提升 per-pattern performance。剛開始增加 patterns 可以改善 SM utilization 與 latency hiding；當 GPU 已經接近飽和時，再增加 patterns 只會讓總工作量增加，per-pattern time 會趨於穩定，甚至因 memory/cache pressure 上升而變差。

8.9 gem5 simulated time 與 CUDA runtime 的比較邊界

Part 1 至 Part 3 的 simSeconds 是 gem5 TimingSimpleCPU 模擬出的時間，代表指定 microarchitecture model 下的 simulated execution time。Part 4 與 Part 5 的 ms 則是實際 NVIDIA GPU 上的 CUDA kernel runtime。這兩種數字不能直接相除，也不應該說 CUDA 比 gem5 P1 快多少倍。正確做法是：gem5 內部比較 scalar/RVV/SIMD-like RVV；CUDA 內部比較 GPU kernel 與 native CPU reference。

9. 結論

我們以 Bilateral Filter 觀察不同 architecture parallelism 的效能行為。Scalar baseline 結構最直接，但每個 neighbor 都由 scalar instructions 逐一處理，instruction count 與 cycle count 最高。

RVV vector reduction 能把同一個 output pixel 的 81 個 neighbors 改成 vector chunks，instruction count 確實下降 9.43%。然而，reduction、vector setup、temporary gather 與 division latency 抵消了大部分 cycle benefit，cycle count 只下降 0.45%。

SIMD-like RVV 的 mapping 更適合本演算法，因為 output pixels 彼此獨立。每個 vector lane 處理一個 output pixel，不需要 reduction，雖然 strided access 讓 D-cache miss rate 稍微上升，整體 cycles 仍下降 62.42%，相對 scalar speedup 約 2.66x。

CUDA SIMT 讓一個 thread 對應一個 output pixel，能自然利用 GPU massive threading。Part 4 對單一 pattern 的 kernel time 是 0.106496 ms，相對 native CPU reference speedup 為 149.21x。Part 5 把 32 個 independent patterns 放入 2D grid，active threads 提升到 547,840，使 GPU latency hiding 與 utilization 更好；per-pattern kernel time 降到 0.06416 ms，對 Part 4 per-pattern time 改善 1.66x。

整體來看，Bilateral Filter 的最佳平行化方向不是只把 single-output summation vectorize，而是把 independent outputs 或 independent patterns 暴露給硬體。RVV 上是 across-output SIMD-like mapping 較有效；CUDA 上則是增加可排程的 blocks/warps，讓 GPU 有足夠工作量去 hide latency。